

CLASH OF THE CLAWS

ClawSwarm AI

Intelligence that doesn't just execute
it evolves.

Team Members:

Pratyush Mohanty
Harsha Sah
Amisha Priya
Almas H.

Team Name:
Photons

Problem Statement

The Rigidity Bottleneck in Multi-Agent Systems

Why We Chose This Theme:

GenAI Research + Productivity

Every existing multi-agent AI framework — CrewAI, AutoGen, LangGraph, MetaGPT — shares one critical flaw: you must decide what every agent does before the system ever runs a single task.

- **Wrong team composition = pipeline bottleneck**
- **Task demand shifts = system can't adapt**
- **One agent fails = entire workflow collapses**

Who faces it: AI developers, researchers, and enterprise teams building complex LLM workflows.

Why it matters: Static roles destroy the autonomy that AI promises.



Our Solution ClawSwarm AI

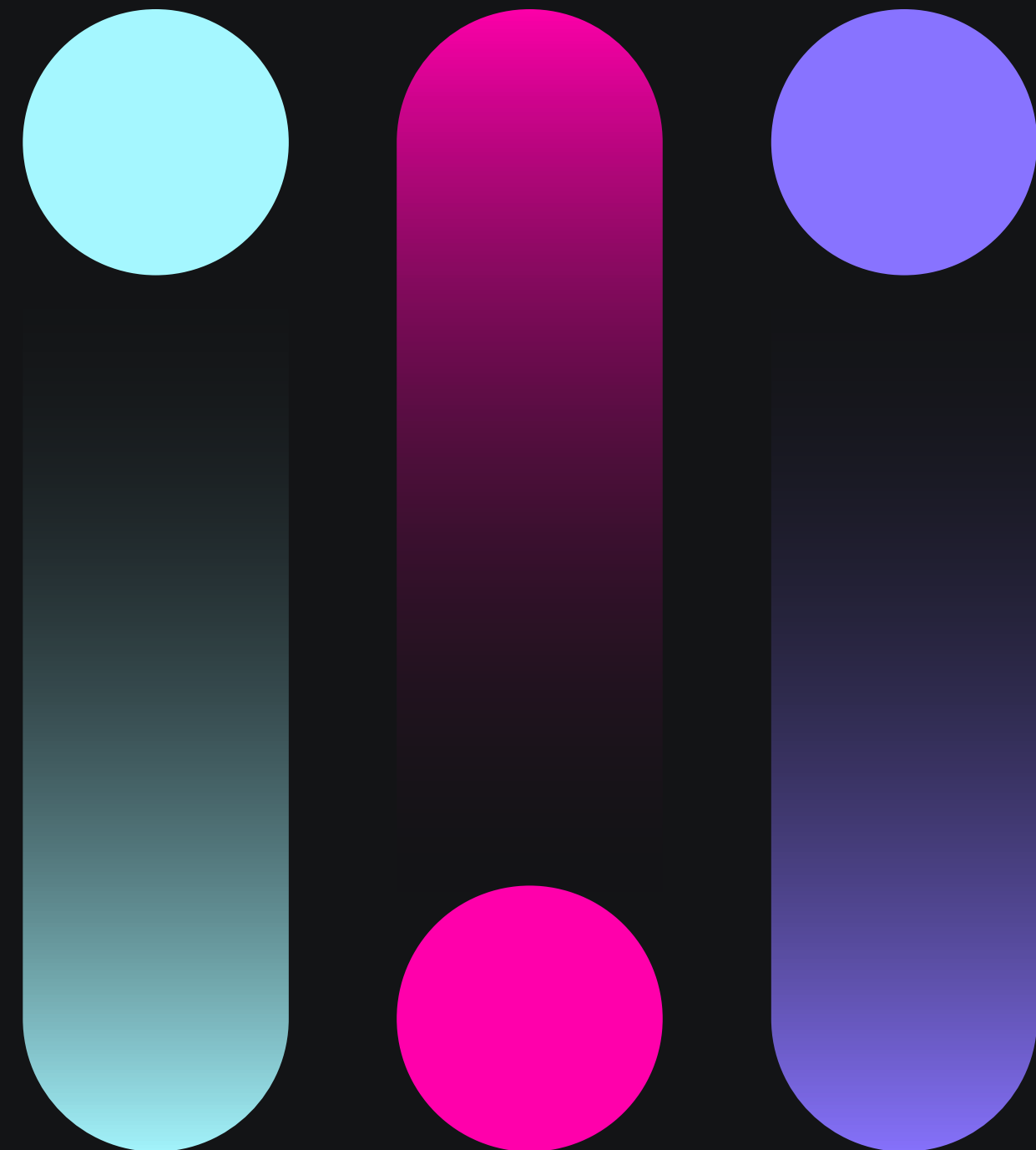
What we built: ClawSwarm AI: a multi-agent LLM framework with Emergent Role Specialization (ERS), a completely new algorithm, never implemented before.

How it works:

1. All agents start identical (skill score: 0.5 across 6 dimensions)
2. Tasks routed via Epsilon-Greedy algorithm (80% best agent, 20% random exploration)
3. An LLM Judge scores every output (0.0–1.0)
4. Scores update agent skill vectors via Exponential Moving Average
5. Roles emerge from math "Specialist: Coding", "Specialist: Research", never configured

Key Features:

- Novel ERS Algorithm
- Epsilon-Greedy Routing
- Sub-2s Fault Tolerance
- Live D3.js Swarm Visualization



Our Solution

ClawSwarm AI

Product Framework	Roles	Adapts?	Self-Heals?
CrewAI	Hardcoded	NO	NO
AutoGen	Hardcoded	NO	NO
LangGraph	Fixed nodes	NO	YES
ClawSwarm	Emergent	YES	YES

Gap: No framework lets agents learn and specialize from their own performance.

Opportunity: Build an AI team that organizes itself just like humans do.

Demo / Product Walkthrough

User types: "Research quantum computing and write a blog post with code examples."

- 1 Decompose → Orchestrator splits into 5 subtasks with dependency order
- 2 **Route** → Epsilon-Greedy sends each task to the current best-fit agent
- 3 **Execute** → Agents run in parallel; dependencies respected
- 4 **Score** → LLM Judge scores output → EMA updates skill vector live
- 5 **Emerge** → Agent role label flips: "Generalist" → "Specialist: Research" (live on dashboard)
- 6 **Synthesize** → Leader combines all outputs → Final result delivered



01

Technical Implementation: The Engine

The Mathematics of Emergent Intelligence:

- **Epsilon-Greedy Routing:** 80% Exploit (use the best specialist), 20% Explore (force a generalist to try a new skill to prevent stagnation).
- **Exponential Moving Average (EMA):** Skill vectors update dynamically:
$$\text{new_score} = (\alpha * \text{performance}) + ((1 - \alpha) * \text{old_score}).$$
- **6-Dimensional Skill Space:** Mathematically tracking agent competency across divergent LLM capabilities.
- **Tech Stack:** Python (AsyncIO), WebSockets, D3.js (Visualization), Redis (Message Bus).

02

Resilience & Fault Tolerance The Self-Healing Swarm

Zero-Downtime Guarantee:

- **Sub-2s Heartbeat Detection:** Continuous monitoring for agent silence or rate-limiting.
- **Automatic Reassignment:** If an agent fails mid-task, the router instantly re-routes to the next best specialist with full context inheritance.
- **Leader Failover:** Sub-1s election of a new Leader if the current one drops.
- **Result:** Reliable execution even in unstable API environments (Groq/Gemini/Anthropic fallbacks).

03

Differentiation

Every existing framework assigns roles. ClawSwarm is the first where roles emerge from performance proven by math.

MOAT 1: Emergent Role Specialization Algorithm

- No competitor (CrewAI, AutoGen, MetaGPT) has this
- System gets smarter with every task — competitors stay static forever
- Verifiable, visible, and proprietary algorithmic IP

MOAT 2: Zero-Downtime Self-Healing

- Sub-2s fault detection + automatic task reassignment
- Score-weighted leader election no hardcoded priority
- The only multi-agent LLM framework that is enterprise-production-safe

Tech & Architecture

Stack:

- **Backend:** Python, FastAPI, asyncio
- **LLMs:** Anthropic Claude 3.5 (primary) → Groq Llama 3 → Gemini 2.0 (fallbacks)
- **Frontend:** React + D3.js (live swarm visualization)
- **Infra:** Docker, Redis pub/sub, WebSocket

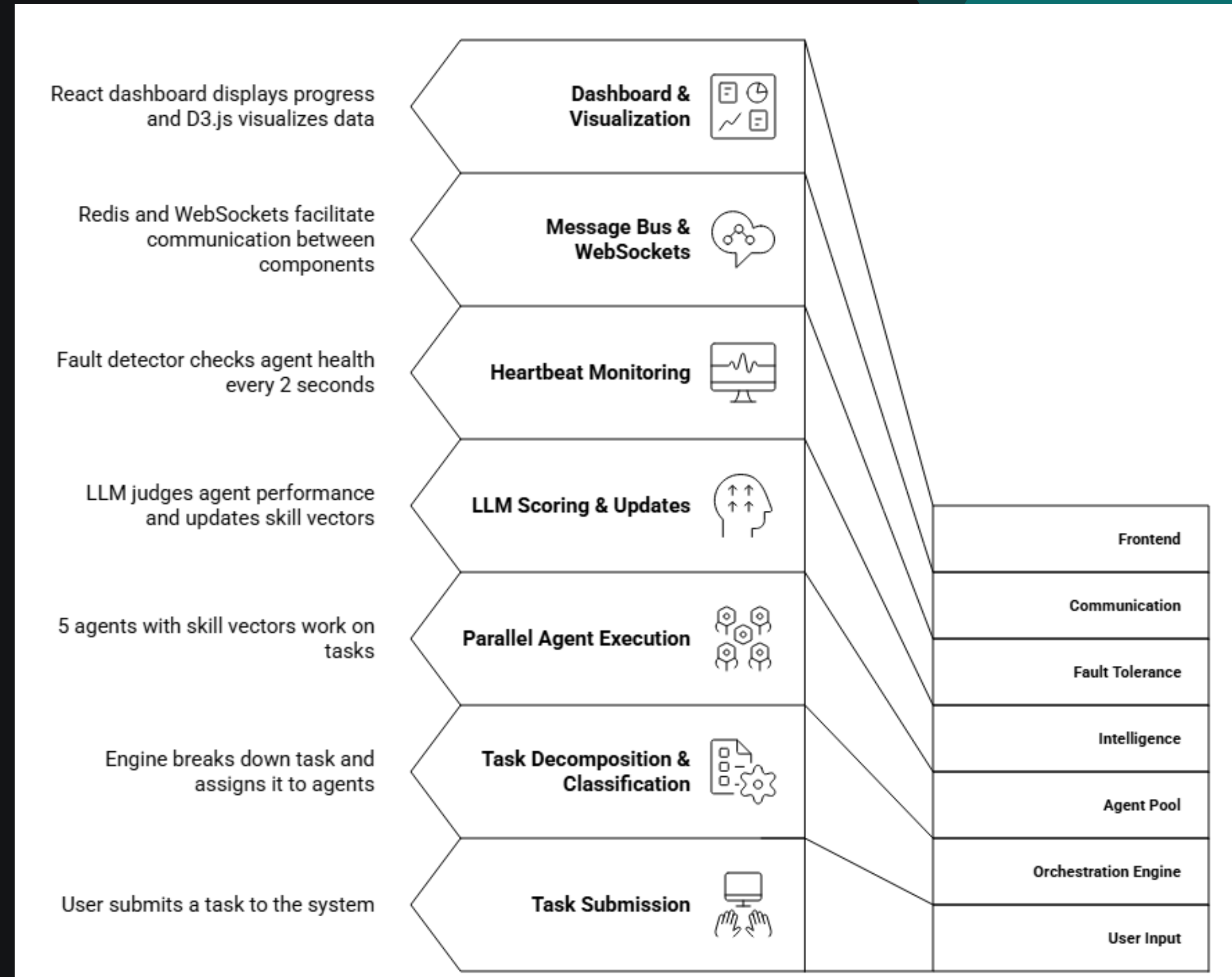
The Novel Math:

- **Skill Update (EMA):**
$$S_{\text{new}} = \alpha \times \text{LLM_score} + (1-\alpha) \times S_{\text{old}}$$
- **Router Decision:**
if $\text{rand}() < 0.2 \rightarrow \text{explore (random agent)}$
else $\text{exploit (best scorer)}$
- **Leader Election:**
$$L = 0.6 \times \text{avg}(\text{SkillVector}) + 0.4 \times \text{success_rate}$$

System Flow: User Task → Decompose → Classify → Route
→ Execute → Score → Evolve → Synthesize

Methodology

The ClawSwarm architecture enables intelligent multi-agent task execution through task decomposition, parallel processing, monitoring, and visualization. Its layered design ensures efficient, scalable, and fault-tolerant AI automation.



Demo Link

GitHub:

<https://github.com/Pratyush426/clawswarnam>

Live Demo:

<https://clawswarnam.vercel.app/>

- `git clone [repo] && cp .env.example .env`
- `docker compose up`
- `# Live at localhost:3000`

Docs submitted:

- `BACKEND_ARCHITECTURE.md`
- `FRONTEND_ARCHITECTURE.md`
- `AI_DISCLOSURE.md`
- `README.md + docker-compose.yml`

Brownie Slide

“ClawSwarm is a research-grade contribution.”

The Emergent Role Specialization (ERS) algorithm sits at the intersection of:

- Multi-Armed Bandit theory (epsilon-greedy)
- Time-series adaptive estimation (EMA)
- Cooperative multi-agent systems

Tasks	Avg Specialization Score	State
0	0	All Generalists
20	0.14	Roles emerging
100	0.35+	Full specialists

"To our knowledge, no published system combines EMA skill vectors with epsilon-greedy routing in a live LLM agent swarm. This is a novel algorithmic contribution with academic publication potential."



Thank You

In the Clash of Claws, the smartest swarm wins
Team PHOTONS